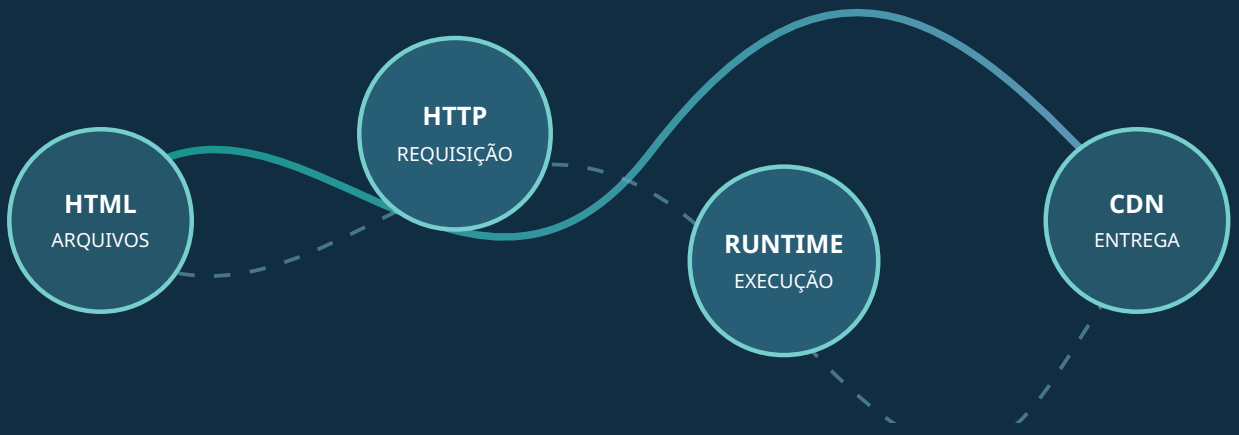
UM GUIA SEM BUZZWORDS

Hospedagem de sites em 2026

Do Apache e do *index.html* à CDN, containers, serverless e edge.



EDIÇÃO · 16 OUTUBRO 2026

24 CAPÍTULOS
4 LABORATÓRIOS
FONTES OFICIAIS

Um livro para entender a tecnologia por trás dos nomes — com exemplos reais, diagramas, um glossário e um caminho de estudo.

NOTA AO LEITOR

Você não perdeu a capacidade de hospedar sites. A indústria só criou mais caminhos.

Se você aprendeu que um navegador chama um endereço, um servidor recebe a requisição e devolve arquivos ou HTML gerado por código, você aprendeu a base certa.

O objetivo deste livro é atualizar esse mapa mental sem jogar fora seu conhecimento anterior. Você vai reconhecer o Apache por trás das abstrações, entender o que realmente fazem Vercel, Netlify, Cloudflare, Docker e AWS, e saber escolher a solução mais simples para cada aplicação.

> **"Toda arquitetura moderna pode ser explicada como arquivos, execução, dados, rede e uma forma de publicar."**

Para quem é	Quem conhece HTML, PHP, Linux ou servidores clássicos e quer alcançar a web atual sem hype.
Como ler	Capítulos 1 a 8 para orientação; 9 a 14 para infraestrutura; 15 a 20 para casos e oficinas; 21 a 24 para consulta.
Escopo	Conceitos, arquiteturas, exemplos e comandos didáticos; não é manual de operação irrestrita de produção.
Atualização	Documentação consultada até 16 de outubro de 2026. Limites, interface dos painéis e preços podem mudar.

MAPA DE LEITURA

Sumário

Clique num capítulo para navegar no PDF. Os números à direita indicam a página de abertura.

I · As fundações

01 A internet continua funcionando do mesmo jeito 4

02 Quatro camadas, dezenas de nomes 6

II · Os modelos de hospedagem

03 A hospedagem clássica: Apache, PHP e FTP 8

04 VPS: o velho servidor, só que seu 10

05 Sites estáticos, CDN e hospedagem moderna 11

06 Git, build e deploy automático 13

07 React, Vue e Next.js: onde cada coisa roda? 15

08 Backend, APIs, bancos e dados persistentes 17

III · Infraestrutura moderna

09 Containers, Docker e PaaS 19

10 Serverless e Functions 21

11 Edge Computing e Cloudflare Workers 23

12 AWS, Google Cloud e o cardápio de infraestrutura 25

13 DNS, domínio, HTTPS e reverse proxy 27

14 Custos, segurança e operação 29

IV · Aplicando de verdade

15 Oito projetos reais — e onde eu hospedaria 31

16 Matriz de decisão sem fanatismo 32

V · Oficinas práticas

17 Laboratório 1: um site estático no Linux Mint 34

18 Laboratório 2: deploy estático moderno 36

19 Laboratório 3: seu próprio Nginx em uma VPS 38

20 Laboratório 4: frontend + API, sem vazar segredos 40

VI · Referência de mesa

21 Quando algo quebra: diagnóstico por sintomas 42

22 Glossário de sobrevivência web 44

23 Roteiro de estudo de 30 dias 45

24 Resumo de uma página: cole ao lado do monitor 46

Após os 24 capítulos, veja as fontes e links de leitura complementar.

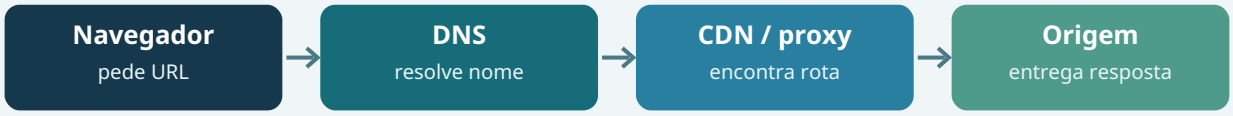
I · AS FUNDAÇÕES

CAPÍTULO 01 / O mapa mental que coloca ordem na bagunça

A internet continua funcionando do mesmo jeito

> **A tese deste livro:** um navegador pede um recurso; alguma coisa entrega uma resposta. A indústria multiplicou as maneiras de construir, executar e distribuir essa resposta. Não mudou a natureza do pedido.

Quando você colocava `index.html` em `/var/www/html` e deixava o Apache trabalhar, possuía um modelo mental correto. Ele continua correto em 2026. O que mudou é que, em muitos projetos, **a pasta publicada não é a pasta onde você escreve o código**, **o servidor de aplicação não é o servidor HTTP**, e **a máquina que responde ao usuário pode não ser aquela em que o projeto foi implantado**.



Um mesmo site pode pular a CDN ou ter várias origens.

Uma requisição real, sem misticismo

Imagine que alguém visita `https://exemplo.com/produtos`:

1. O navegador descobre qual endereço de rede atende aquele domínio (DNS).
2. Estabelece uma conexão segura, normalmente via HTTPS/TLS.
3. Envia uma requisição, como `GET /produtos`.
4. Algum componente decide de onde vem a resposta: um arquivo em cache, um arquivo na origem ou código que cria HTML/JSON.
5. O navegador interpreta HTML e busca CSS, JS, fontes e imagens adicionais.

Em muitos sites há **uma cadeia de vários serviços**; em outros, apenas um Nginx. Nos dois casos, a linguagem comum é HTTP. O servidor pode responder `200 OK`, `301`/`308` (redirecionamento), `404` (não encontrado) ou `500` (erro interno).

Cinco perguntas suficientes para entender qualquer hospedagem

- **O que é entregue?** Arquivos prontos, HTML criado dinamicamente, JSON, mídia?
- **Onde o código é executado?** Navegador, servidor dedicado, container, função, edge?

- **Onde o estado mora?** Banco de dados, arquivos, objetos, cache ou nenhum?
- **Quem faz o deploy?** Você copia arquivos ou uma plataforma compila e publica?
- **Quem opera a infraestrutura?** Você, a plataforma ou um pouco de cada?

Essas perguntas são muito mais úteis do que decorar nomes comerciais. A descrição clássica do servidor web da MDN continua sendo um ótimo ponto de partida S01.

As palavras que confundem

Hospedagem é disponibilizar a aplicação para o público. **Build** é transformar código-fonte em artefatos de produção. **Deploy** é colocar uma versão em funcionamento. **Runtime** é o ambiente em que o código executa. **CDN** replica ou mantém cache de conteúdo perto do usuário. **Framework** organiza como o programa é construído: não é, por si só, um lugar para hospedar.

> **Regra de ouro:** React não é servidor. Next.js não é hospedagem. Docker não é nuvem. GitHub não é automaticamente um servidor de aplicação. Todos podem participar do mesmo projeto, mas cumprem papéis distintos.

I · AS FUNDAÇÕES

CAPÍTULO 02 / Tradução simultânea do marketing para a engenharia

Quatro camadas, dezenas de nomes

Em vez de ver uma coleção de marcas, imagine quatro camadas separadas:



Camada visível. HTML, CSS e JavaScript descrevem o que aparece. O navegador os executa ou renderiza. Uma página estática pode viver aqui por inteiro.

Camada de execução. Quando é necessário validar um pedido, consultar um dado privado, criar uma sessão ou processar pagamentos, código deve rodar num ambiente de servidor. Antigamente isso frequentemente significava PHP + Apache; hoje pode ser PHP-FPM, Node, Python, Go, um container ou uma função.

Camada de dados. Uma aplicação pode gravar usuários em PostgreSQL, imagens em object storage, mensagens em filas, sessões em cache. Não confunda os arquivos usados para publicar o aplicativo com os dados de usuários, que precisam sobreviver a novas versões.

Camada de infraestrutura. É quem oferece CPU, memória, conectividade, DNS, armazenamento e rede. Pode ser a sua VPS, uma hospedagem compartilhada, uma plataforma gerenciada ou vários serviços.

Um pequeno dicionário comparativo

Você conhecia	Nome que aparece hoje	O que realmente significa
Pasta `public_html`	`public/`, `dist/`, `build/`, `out/`	Local de arquivos publicados, às vezes gerados por build
Subir por FTP	`git push`, CI/CD, CLI deploy	Processo automatizado de publicação

Você conhecia	Nome que aparece hoje	O que realmente significa
PHP ao acessar página	Server-side rendering, função, API	Código processado antes de responder
Apache como porta de entrada	Nginx, proxy, load balancer, edge	Componente que recebe/encaminha HTTP
Pasta de uploads	Bucket S3/R2, blob storage	Armazenamento durável de objetos
Um servidor e MySQL	App + managed database	Executar código e persistir dados separados

Por que isso ficou tão complexo?

Porque há objetivos que entram em conflito: **simplicidade, custo, escalabilidade, distribuição geográfica, segurança, liberdade tecnológica e carga operacional**. Empresas criaram produtos para otimizar combinações diferentes dessas dimensões. O problema é quando se vende a solução de uma megacorporação como requisito para um portfólio de três páginas.

O que você precisa realmente aprender

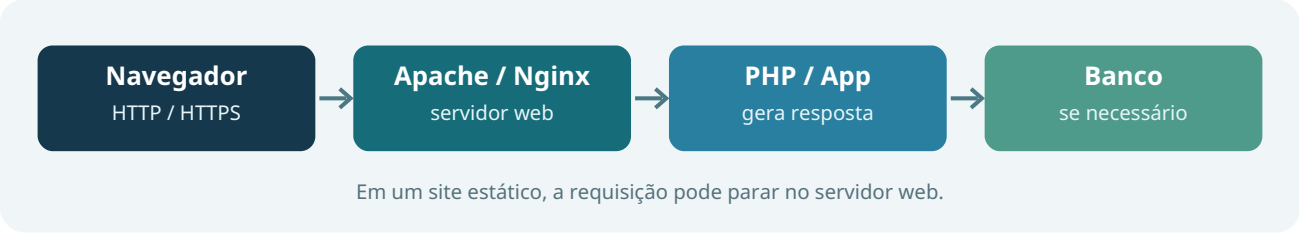
É suficiente dominar: (a) arquivos estáticos e HTTP; (b) execução no servidor; (c) dados persistentes; (d) DNS + HTTPS; (e) como uma versão nova chega à produção. Kubernetes, service mesh, multi-cloud e orquestração distribuída podem ficar para necessidades que realmente os justifiquem.

II · OS MODELOS DE HOSPEDAGEM

CAPÍTULO 03 / Sim, ela está viva e às vezes é a melhor escolha

A hospedagem clássica: Apache, PHP e FTP

No modelo clássico, você compra um plano de hospedagem compartilhada, recebe um diretório — frequentemente `public_html` —, um painel como cPanel, acesso por SFTP/FTP e, conforme o plano, PHP e MySQL/MariaDB.



Em um site estático, a requisição pode parar no servidor web.

Como fica um projeto

```
public_html/
├── index.html
├── produtos.php
├── css/
│   └── style.css
└── img/
    └── logo.png
```

Quando alguém visita `/`, o servidor costuma procurar um arquivo de índice. Quando visita `/produtos.php`, o servidor passa o processamento à integração com PHP; o código PHP **roda no servidor** e entrega o resultado, não o código-fonte, ao navegador.

```
<?php
$nome = "Mundo";
echo "<h1>Olá, {$nome}!</h1>";
```

Usos reais

- **Site de escritório ou de comércio local:** páginas institucionais, formulários simples e área administrativa básica.
- **WordPress:** CMS em PHP com banco de dados; hospedagem tradicional segue sendo um encaixe natural.

- **Sistema interno herdado:** app PHP tradicional bem entendido pela equipe; migrar só por moda pode piorar tudo.
- **Site estático minúsculo:** transferir três arquivos continua sendo perfeitamente aceitável.

Prós e contras

Ganhos: previsibilidade, pouco ferramental, ampla documentação, custo inicial acessível, PHP pronto e painéis amigáveis. **Trocas:** vizinhos dividindo recursos; restrições de processo, versões e portas; escalabilidade limitada; variabilidade de performance e de suporte.

FTP, SFTP e segurança

Evite FTP sem proteção: ele transmite credenciais e dados em texto claro. Prefira **SFTP** (via SSH) ou outro método explicitamente cifrado. Use HTTPS no site, contas com o menor privilégio possível, atualizações do CMS/plugins e backups testados. E não ponha senhas no diretório publicável.

> **Quando escolher:** se seu projeto precisa de PHP/WordPress, você não quer gerenciar Linux e não tem necessidades incomuns de processamento ou tráfego, hospedagem tradicional pode ser a opção mais sensata.

II · OS MODELOS DE HOSPEDAGEM

CAPÍTULO 04 / A opção mais transparente para quem gosta de Linux

VPS: o velho servidor, só que seu

Uma **VPS (Virtual Private Server)** é uma máquina virtual com sistema operacional, disco, CPU, memória e rede próprios dentro da infraestrutura de um provedor. Você acessa via SSH, instala Nginx/Apache, escolhe linguagens, cria serviços systemd e decide onde ficam os arquivos.

O que acontece em produção

```
Internet
  └── domínio + DNS
        └── IP da VPS
              └── Nginx :443
                    ├── /var/www/site/     (arquivos estáticos)
                    └── proxy_pass :3000  (aplicação Node/Python/Go)
                          └── PostgreSQL / API / arquivos
```

Nginx pode servir `index.html` diretamente **e** encaminhar `/api/` para um processo que escuta em `127.0.0.1:3000`. O navegador não precisa enxergar a porta interna.

Para que é excelente

- Um dashboard próprio com API e banco de dados.
- Um serviço de integração, webhook, bot com painel web e jobs em segundo plano.
- Uma aplicação com WebSockets, conexões persistentes ou processos longos.
- Hospedar vários projetos pequenos no mesmo servidor com domínios distintos.

O preço não é só a mensalidade

Na VPS você cuida de atualizações, firewall, proteção de credenciais, TLS, backups, logs, restauração e incidentes. Reinicializações e mudanças de dependência são parte do trabalho. A VPS não é automaticamente mais barata quando se considera o tempo operacional.

Três cuidados fundamentais

1. **Exponha apenas o necessário:** portas 80/443 para web e SSH cuidadosamente protegido; banco de dados não precisa ficar aberto ao mundo.
2. **Separe processo e dados:** o deploy não deve apagar banco, uploads, logs importantes ou segredos.
3. **Prepare a recuperação:** snapshot não substitui necessariamente backup independente; teste restore.

II · OS MODELOS DE HOSPEDAGEM

CAPÍTULO 05 / Seu index.html foi promovido — não extinto

Sites estáticos, CDN e hospedagem moderna

Um site **estático** entrega arquivos já preparados: HTML, CSS, JavaScript e mídia. Isso inclui páginas manuscritas, documentação, geradores como Hugo ou Astro em modo estático e aplicações React/Vue construídas para rodar no navegador.

A pasta publicada pode ser exatamente a que você escreveu ou o resultado de um comando, como `npm run build`. Em muitos projetos essa pasta chama-se `dist/`. **O navegador continua recebendo arquivos!**

O que é a CDN, afinal?

Uma **Content Delivery Network** mantém cópias, caches ou caminhos otimizados para servir arquivos a usuários em várias regiões. Ela pode atender uma imagem sem conversar com o servidor de origem, reduzindo latência e carga. Nem todo arquivo é cacheado; o comportamento depende de cabeçalhos e regras.

```
Você escreve:  src/App.jsx + CSS + assets
Você constrói: npm run build
Você publica:  dist/index.html + dist/assets/*.js + *.css
Pessoa acessa: CDN entrega HTML, JS e CSS
```

Opções com perfil diferente

Serviço	Bom para	Atenção
GitHub Pages	Portfólio, docs, projetos públicos	Não executa PHP/backend; políticas de uso específicas
Netlify	Estáticos, previews, funções leves	Cotas e modelo de cobrança variam
Vercel	Frontends e Next.js	Planos e limites, inclusive uso comercial
Cloudflare Pages	Projetos existentes e deploy Git	Cloudflare recomenda Workers para **novos** projetos em 2026
Cloudflare Workers Static Assets	Estáticos e apps híbridos novos	Runtime e modelo de implantação próprios
AWS Amplify Hosting	Frontend no ecossistema AWS	A conta AWS exige atenção à cobrança

Importante, outubro de 2026: a documentação da Cloudflare coloca **Workers Static Assets** como caminho preferencial para novos sites, SPAs e projetos full-stack. Pages permanece disponível, incluindo Git e upload direto S02, S04.

Usos reais

Um manual de engenharia, um blog sem login, uma landing page, um site de conferência ou um portfólio costumam precisar somente de arquivos estáticos. Se houver formulário, ele pode chamar uma API ou serviço externo. O site permanece estático; o **sistema completo** passa a ter uma parte dinâmica.

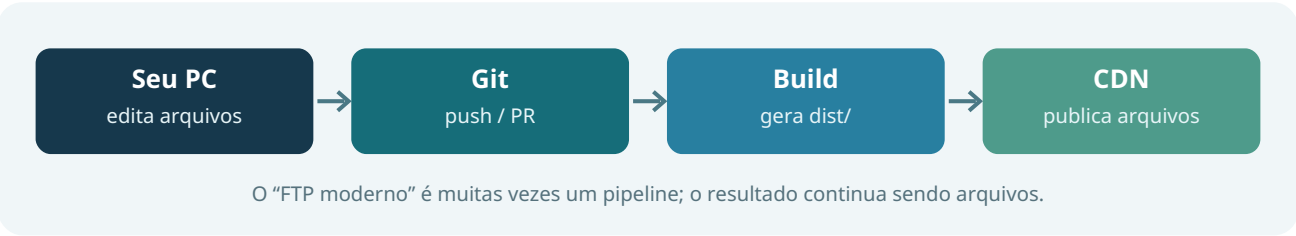
> **Regra prática:** quando nenhuma lógica precisa executar no servidor a cada visita, não contrate um servidor de aplicação só para servir HTML.

II · OS MODELOS DE HOSPEDAGEM

CAPÍTULO 06 / Entendendo o novo “arrastar arquivos para o servidor”

Git, build e deploy automático

Git não serve o site aos visitantes. Ele guarda versões do código. Uma plataforma ligada ao repositório observa mudanças e executa uma receita de publicação. Isso é **CI/CD** no nível mais acessível: automatizar testes, construção e entrega.



O “FTP moderno” é muitas vezes um pipeline; o resultado continua sendo arquivos.

Exemplo concreto

Suponha um projeto com `src/`, `package.json` e `public/`. O servidor de hospedagem recebe uma instrução: **comando de build `npm run build`; diretório publicado `dist`**. O provedor prepara um ambiente temporário, instala dependências, compila e publica o conteúdo de `dist`.

```
git add .
git commit -m "Atualiza página de contato"
git push origin main
```

Não foi o `git push` que transformou magicamente o navegador. Ele **disparou um pipeline**; o produto final é um conjunto de arquivos ou um pacote executável.

Quatro termos úteis

Branch: linha paralela de trabalho. **Pull request (PR):** proposta de alteração revisável. **Preview deploy:** endereço temporário para inspecionar a versão da branch. **Rollback:** retornar a um deploy anterior. A Cloudflare Pages documenta integração com GitHub/GitLab e previews S05; a Netlify descreve deploy por Git, CLI ou upload S07.

Build vs execução

A build pode usar Node.js para transformar React em arquivos estáticos. Isso **não implica** que Node.js rode no servidor para cada visita. Já um framework que depende de SSR pode precisar de runtime em produção. Essa distinção elimina metade do ruído.

E se eu não quiser Git?

Sem drama: serviços como Netlify permitem publicação por upload; Cloudflare Pages oferece Direct Upload, e Workers permite deploy via ferramenta CLI. Git é excelente para repetibilidade e histórico, **não um requisito universal da web** S07, S23.

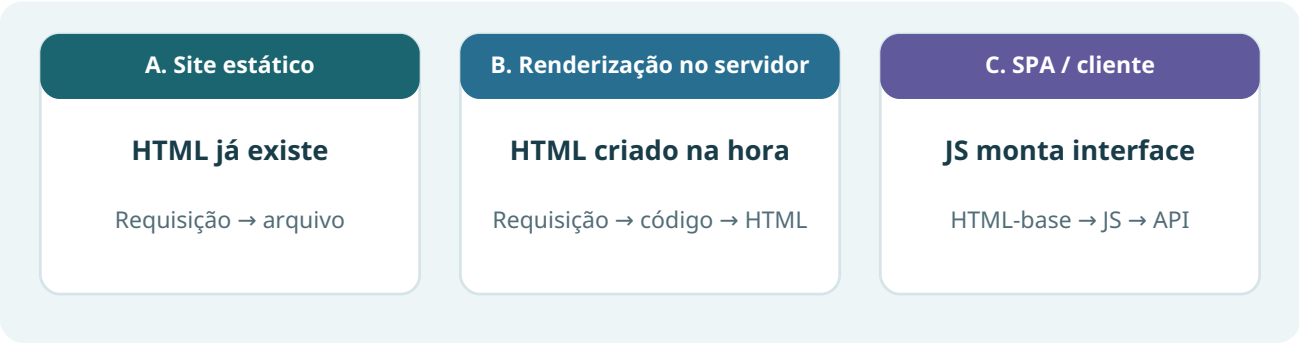
> **Exemplo de projeto real:** uma equipe publica documentação. Cada PR cria uma prévia, revisores aprovam e o merge dispara o deploy. O visitante continua lendo HTML — só o processo editorial ficou melhor.

II · OS MODELOS DE HOSPEDAGEM

CAPÍTULO 07 / Diferencie biblioteca, framework e plataforma

React, Vue e Next.js: onde cada coisa roda?

Há duas perguntas independentes: **como a interface é construída?** e **onde o HTML é produzido?** React e Vue são ferramentas para interfaces; Next.js é um framework que organiza diversas estratégias de renderização e execução.



O espectro da renderização

HTML estático escrito à mão. O arquivo já existe no disco. Ideal para materiais simples, confiáveis e fáceis de manter.

SSG (Static Site Generation). Uma build cria as páginas antecipadamente. Um blog de 100 artigos pode produzir 100 HTMLs sem executar o gerador em cada visita.

CSR/SPA (Client-Side Rendering / Single Page Application). O servidor entrega uma página-base e JavaScript. O navegador carrega e monta a interface, frequentemente consultando APIs. O servidor web pode continuar sendo puramente estático.

SSR (Server-Side Rendering). Código no servidor cria HTML no momento da requisição, talvez lendo uma sessão e dados. Exige infraestrutura que rode o código de servidor; pode haver cache e combinações com estático.

Híbrido. Uma mesma aplicação mistura páginas estáticas, dinâmicas e componentes que executam no navegador. É comum em Next.js e frameworks modernos.

Mesma aplicação, três entregas

Aplicação	Como sai o HTML	Onde publicar
Portfólio React compilado como SPA	JS monta no browser	CDN / hospedagem estática

Aplicação	Como sai o HTML	Onde publicar
Blog Astro/Hugo com SSG	HTML criado na build	CDN / hospedagem estática
Painel Next.js com SSR	HTML em requisição	Node, container ou plataforma compatível

O Next.js documenta execução como **servidor Node.js, container Docker, exportação estática ou adaptadores**, com diferentes conjuntos de recursos. Exportação estática não suporta tudo que o servidor Node oferece S10.

A armadilha clássica: refresh da rota SPA

Seu frontend faz `/painel/relatorios` funcionar por navegação interna; ao pressionar F5, o servidor procura um arquivo `/painel/relatorios` e retorna 404. É preciso configurar fallback adequado para `index.html` **sem encobrir arquivos ausentes ou rotas de API**.

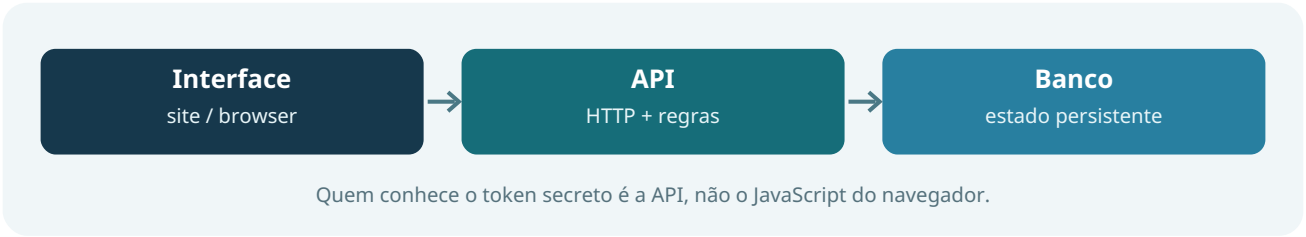
> **Moral:** framework não escolhe infraestrutura sozinho; os recursos efetivamente usados determinam se os arquivos estáticos bastam ou se há código de servidor a executar.

II · OS MODELOS DE HOSPEDAGEM

CAPÍTULO 08 / A fronteira onde começam as responsabilidades reais

Backend, APIs, bancos e dados persistentes

Um site com login, pedidos, mensagens ou informação privada costuma precisar de um **backend**. Ele recebe chamadas HTTP, autentica quem pediu, valida dados, aplica regras e conversa com serviços internos.



Quem conhece o token secreto é a API, não o JavaScript do navegador.

Exemplo: dashboard para um serviço de mensagens

A tela HTML/React pode morar numa CDN. Ela faz `GET /api/status` ou `POST /api/comandos`. A API roda na VPS ou em uma plataforma e consulta banco, filas e serviços externos. O segredo de acesso do webhook **não** vai para JavaScript que o navegador baixa; ele permanece no backend.

```
Frontend público (CDN)       Backend (VPS/PaaS)
index.html + app.js    --->   GET /api/status
                              ├── autenticação
                              ├── regra de negócio
                              ├── PostgreSQL
                              └── serviço externo
```

O que é estado?

Estado é a informação que precisa sobreviver: conta do usuário, publicação, pedido, foto enviada, pagamento, fila de tarefa. **Disco efêmero**, comum em ambientes de função e em algumas plataformas, não deve ser a única cópia de um upload.

Banco relacional: PostgreSQL/MySQL; forte para relacionamentos, transações e consultas. **Object storage:** arquivos e mídia; S3 ou compatíveis. **Cache/Redis:** dados temporários e acesso rápido; não substitui banco automaticamente. **Fila:** trabalhos assíncronos como envio de emails.

Coisas fáceis de esquecer

- **Migrações de schema:** a versão nova precisa ser compatível com a base.
- **CORS:** controla quais origens podem ler respostas no navegador; não é autenticação.

- **Cookies/sessões:** atenção a `Secure`, `HttpOnly` e `SameSite`.
- **Secrets:** variáveis secretas e gerenciadores de segredo, jamais versionados no repositório público.
- **Backup:** cópia recuperável e ensaio real de restauração.

Arquiteturas simples funcionam

Se a aplicação é pequena, uma VPS com Nginx + backend + PostgreSQL pode ser mais fácil de entender e até operar do que cinco provedores. Se os requisitos de disponibilidade e crescimento aumentarem, separe banco e aplicação quando houver **motivo concreto**.

III · INFRAESTRUTURA MODERNA

CAPÍTULO 09 / Empacotar aplicações não significa abandonar servidores

Containers, Docker e PaaS

Container é um processo isolado, empacotado com boa parte de suas dependências. Diferentemente de uma VM completa, em geral compartilha o kernel do sistema hospedeiro. **Docker** é um conjunto de ferramentas para construir e executar essas imagens S13.

```
Código do app + dependências
       │
       └── Dockerfile → imagem versionada
                              │
                              ├── container na sua VPS
                              └── container em plataforma gerenciada
```

Por que usar?

Pense num backend Python que requer versões específicas de bibliotecas ou um aplicativo Node com dependências nativas. A imagem especifica um ambiente consistente. Pode ser executada na sua máquina Linux, em uma VPS ou numa plataforma compatível.

```
FROM python:3.13-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["python", "app.py"]
```

Exemplo ilustrativo: um projeto real deve definir usuário sem privilégios, versões imutáveis, health checks quando apropriados e evitar embutir segredos na imagem.

Docker Compose

É um arquivo declarativo para executar **vários serviços relacionados** (app, banco, cache etc.). Ele substitui um conjunto de comandos manuais. Não é automaticamente um orquestrador multi-servidor ou solução completa de alta disponibilidade S14.

```
services:
  web:
    build: .
    ports:
      - "127.0.0.1:3000:3000"
    restart: unless-stopped
```

Esse exemplo limita o acesso direto à porta 3000 à interface local da VPS; um reverse proxy pode publicá-lo pela porta 443. O aplicativo dentro do container precisa escutar na interface apropriada (normalmente `0.0.0.0`).

E o tal PaaS?

Platform as a Service recebe código ou imagens e cuida de partes do deploy e da operação. Pense em Render, Railway, Fly.io ou serviços gerenciados como AWS App Runner — cada um com políticas próprias. App Runner, por exemplo, faz deploy a partir de código-fonte ou de imagem e gerencia parte da infraestrutura S18.

VPS + Docker: controle alto; você opera host, atualizações e backups. **PaaS:** menos administração de sistema; você depende de limites, custos e abstrações da plataforma.

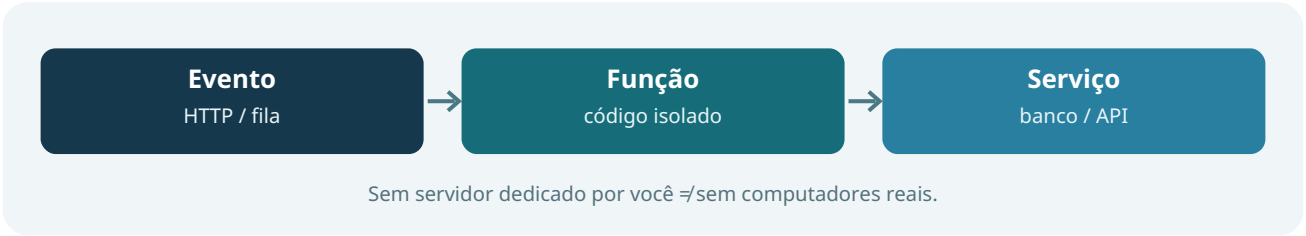
> **Boa escolha:** PaaS para colocar um backend comum no ar sem virar administrador do host. Docker na VPS quando você quer previsibilidade de ambiente e aceita fazer a operação.

III · INFRAESTRUTURA MODERNA

CAPÍTULO 10 / A função não aparece do nada: alguém executa por você

Serverless e Functions

A palavra **serverless** é propaganda com um fundo de verdade: não é ausência de servidor, mas a possibilidade de **não provisionar nem administrar um servidor permanente para aquela execução**. Você implanta código e o provedor aloca o ambiente conforme as regras do serviço.



Exemplo: formulário de contato

Uma landing page estática tem um formulário. Ao clicar em enviar, o browser chama `/api/contato`. Uma função valida os campos e envia a informação para um provedor de email. Não existe a obrigação de manter uma VPS de backend aberta 24 horas por dia só para esse endpoint.

Exemplos de plataforma

- **Vercel Functions:** endpoints e execução ligada ao projeto, com regras de runtime e cobrança.
- **Netlify Functions:** código de servidor publicado junto ao site, com logs e deploys S21.
- **AWS Lambda:** execução baseada em eventos, APIs, arquivos, filas e integrações de cloud S17.
- **Cloudflare Workers:** execução em runtime voltado a eventos e lógica web, com particularidades próprias.

Em outubro de 2026 a AWS também documenta **Lambda Web Functions** para aplicações HTTP que executam servidores web em ambiente serverless. É um recurso distinto das funções tradicionais orientadas a handler, e ilustra como o setor continua evoluindo S27.

As pegadinhas da tecnologia

Inicialização: ambientes podem precisar começar (cold start). **Limites:** memória, CPU, duração, tamanho e concorrência variam por produto e plano. **Estado:** processos podem ser reutilizados, mas você não deve assumir memória/disco persistente. **Custos:** pagar por requisição, computação, transferência ou produtos associados pode ser vantajoso para uso intermitente e caro para outros padrões.

O que costuma funcionar mal

Tarefas contínuas, conexões WebSocket com necessidades específicas, processamento sem limites claros, sessões em memória local, dependências que requerem SO completo ou alto tráfego sustentado podem pedir um serviço permanente. Não é uma proibição universal: o suporte varia por runtime e fornecedor.

> **Tradução para 2005:** em vez de deixar um processo esperando requisições, você entrega uma rotina para a infraestrutura do provedor chamá-la quando houver trabalho.

III · INFRAESTRUTURA MODERNA

CAPÍTULO 11 / Executar perto do usuário — com o pé no chão

Edge Computing e Cloudflare Workers

Edge se refere à execução ou à entrega por uma infraestrutura geograficamente distribuída, mais próxima da borda da rede. Pode reduzir latência em certas operações, mas não elimina a distância de um banco centralizado. Uma requisição pode executar na borda e ainda precisar consultar PostgreSQL a milhares de quilômetros.

Três coisas que não são iguais

CDN: entrega arquivos, cache e roteamento. **Edge function/Worker:** executa lógica perto do usuário, dentro do runtime permitido. **Origem:** serviço de onde vêm dados ou arquivos quando não estão no cache. Um site pode usar os três.

Exemplo 1: personalização leve

Uma pequena rotina lê cabeçalhos e redireciona o visitante para idioma/região apropriados. Por ser tarefa rápida, com poucos dados e sem uma base central consultada a cada requisição, pode combinar com Workers.

Exemplo 2: HTML, CSS, JS e API no mesmo deploy

Cloudflare Workers permite publicar uma pasta de **Static Assets** ao lado de código executado no Worker. Em 2026, sua documentação descreve esse caminho como preferencial para novos projetos, no lugar de iniciar um Pages novo S03, S04.

```
{
  "name": "meu-site-estatico",
  "compatibility_date": "2026-10-16",
  "assets": { "directory": "./public" }
}
```

Esse arquivo é um exemplo de `wrangler.jsonc` /configuração equivalente para **site apenas estático**. Nesse cenário não é necessário ter script de Worker S24. O arquivo de configuração deve estar no diretório do projeto e a ferramenta Wrangler instalada/disponível.

Escolha correta e escolha ruim

Boa: páginas estáticas, personalização rápida, cache, pequenas APIs, validações e regras perto do usuário. **Cuidado:** lógica que depende de disco local, binários incompatíveis com o runtime, tarefas muito longas, limites de CPU ou acesso frequente a uma origem distante. Consulte limites atuais antes de decidir S20.

> **Não confunda “global” com “instantâneo”:** rede distribuída ajuda, mas banco de dados, cache miss, APIs externas, tamanho do JS e qualidade da conexão continuam importando.

III · INFRAESTRUTURA MODERNA

CAPÍTULO 12 / Aprenda o mapa; não tente decorar o restaurante inteiro

AWS, Google Cloud e o cardápio de infraestrutura

Cloud pública oferece serviços em muitas camadas. O problema é a tentação de escolher um serviço para cada preocupação antes de se ter uma aplicação funcional. A AWS é um bom exemplo porque os nomes deixam claro quão diferentes são as decisões.

Traduzindo serviços comuns

Categoria	Exemplo AWS	Função fundamental
Máquina virtual	EC2	Computador que você administra
Objetos e arquivos	S3	Armazenar arquivos/artefatos duráveis
CDN	CloudFront	Distribuir conteúdo com cache e políticas
Funções	Lambda	Executar código por evento/HTTP conforme produto
Contêineres	ECS/Fargate	Executar/gerenciar serviços em containers
Hospedagem gerenciada	Amplify Hosting, App Runner	Simplificar publicação de frontend ou app
Banco gerenciado	RDS	Operar banco relacional com serviços gerenciados

AWS S3 puro não é um servidor PHP. Ele pode disponibilizar objetos como HTML/CSS/JS, mas não executa aplicação da mesma maneira que um runtime web. O endpoint de website do S3, isoladamente, **não oferece HTTPS**; a AWS recomenda caminhos como Amplify Hosting ou S3 + CloudFront com configuração segura S15, S25.

Três arquiteturas de exemplo

Portal de documentação: conteúdo gerado no build → CDN/Amplify → navegador. Sem servidor de aplicação permanente.

API com tarefas assíncronas: frontend estático → API (Lambda/App Runner) → banco gerenciado; jobs de processamento por fila.

SaaS empresarial: frontend + APIs independentes + banco gerenciado + filas + armazenamento de objetos + observabilidade, com controles de segurança e redundância. É justificável por requisitos, não por status.

PaaS não é sinônimo de serverless

No PaaS você pode ter um **serviço persistentemente ativo**, apenas sem administrar a máquina. Em serverless por funções, a unidade operacional é normalmente uma execução ou serviço com escalonamento muito diferente. Uma plataforma pode combinar ambos.

Os equivalentes fora da AWS

Google Cloud e Microsoft Azure oferecem categorias semelhantes, com contratos, nomenclaturas e particularidades distintos. Você não precisa estudar todos simultaneamente. Aprenda o conceito — VM, objetos, CDN, app gerenciado, funções, banco — e só então mapeie para o fornecedor.

III · INFRAESTRUTURA MODERNA

CAPÍTULO 13 / Os componentes invisíveis que fazem um site parecer “mágico”

DNS, domínio, HTTPS e reverse proxy

Domínio é o nome que pessoas digitam; **DNS** contém registros para descobrir a rota; **HTTPS** protege a conexão usando TLS; **reverse proxy** é um servidor que recebe a requisição e a encaminha a serviços internos quando necessário.

A, AAAA, CNAME e outros registros

- **A:** nome → endereço IPv4.
- **AAAA:** nome → endereço IPv6.
- **CNAME:** nome → outro nome DNS (com restrições conforme zona/provedor).
- **MX/TXT:** usados entre outras coisas para email, verificações e políticas.

O registrador do domínio, o provedor DNS e a empresa que hospeda o site podem ser **três organizações diferentes**. Apontar DNS para uma VPS não copia arquivo nenhum: apenas dirige tráfego.

Proxy e HTTPS no desenho



A URL pública esconde detalhes internos. Vários domínios podem compartilhar o mesmo IP e porta, com a configuração adequada de virtual hosts/SNI.

Certificados e Let's Encrypt

Em VPS, uma solução comum é emitir e renovar certificados com Certbot para Nginx ou Apache; o comando exato depende do sistema e modo de instalação. A documentação oficial do Certbot oferece instruções por sistema e servidor S19. Em plataformas gerenciadas, HTTPS e renovação frequentemente são providenciados automaticamente.

O que verificar antes de culpar a hospedagem

1. O nome resolve para o destino esperado?
2. O servidor responde nas portas 80/443?

3. O certificado cobre o domínio e está válido?
4. A rota está configurada no proxy/CDN?
5. O backend que recebe o proxy está ativo?
6. O cache está mostrando uma versão antiga?

Cache ≠ banco

Cache de CDN guarda respostas por políticas de validade, podendo poupar requisições à origem. Banco de dados guarda o **estado autoritativo**. Cache mal configurado pode vazar conteúdo privado ou desatualizar páginas. Não cacheie respostas personalizadas/publicamente sem entender as regras.

III · INFRAESTRUTURA MODERNA

CAPÍTULO 14 / O que não aparece no preço “a partir de zero”

Custos, segurança e operação

O custo de hospedagem é a soma de **infraestrutura, tráfego, serviços anexos, proteção, backups e trabalho humano**. O preço de entrada explica pouco sobre a conta de uma aplicação em produção.

VPS	Servidor e manutenção
PaaS	Compute + serviços
Serverless	Execuções + tráfego
Estático/CDN	Build + armazenamento + egress

Como estimar uma conta sem adivinhar tarifas

Para um site estático, avalie **armazenamento, número de builds, largura de banda/egress, requisições e domínio**. Para um backend, adicione **CPU, memória, horas de serviço, invocações, banco, rede entre serviços e logs**. Para uma VPS, some **servidor fixo + snapshot/backup + eventuais serviços terceiros + tempo de administração**.

```
Custo mensal aproximado = hospedagem + tráfego + banco
                        + armazenamento + backup + observabilidade
                        + serviços externos + margem para picos
```

Tarifas e franquias mudam. Este livro evita números fixos que envelhecem depressa: confira página de preços **e termos de uso** antes de publicar serviço comercial. Vercel documenta que o plano Hobby é voltado a projetos pessoais não comerciais; há limites específicos por produto S22, S08.

Modelo de responsabilidade

Você cuida	Provedor costuma cuidar (dependendo do plano)
Código, bugs, credenciais e regras de acesso	Hardware, data center, rede física
Dados pessoais, backup lógico, chaves de API	Sistema operacional em PaaS/serverless
Permissões do banco e segurança de app	Terminação TLS ou CDN gerenciada

Você cuida	Provedor costuma cuidar (dependendo do plano)
Limites de gasto, logs e incidentes	Escala automática se contratada e suportada

A expressão “gerenciado” não significa “seguro por padrão para qualquer aplicação”.

Checklist operacional mínimo

- Habilite autenticação forte/MFA nos provedores e repositórios.
- Separe segredos de código; mantenha versões de dependências atualizadas.
- Faça backup de **dados persistentes**, de preferência fora do ambiente principal.
- Configure alertas para erro, indisponibilidade e gastos.
- Defina limites de recursos e reduza portas expostas.
- Pratique rollback e restauração **antes** de precisar deles.

> **O barato pode sair caro:** um site estático com poucos acessos pode custar quase nada; uma função sem limite de uso, uma API abusada ou uma consulta de banco cara pode surpreender.

IV · APLICANDO DE VERDADE

CAPÍTULO 15 / Casos concretos com trade-offs explícitos

Oito projetos reais — e onde eu hospedaria

1. Portfólio profissional com contato. HTML/CSS simples ou gerador estático → Workers Static Assets, Netlify ou GitHub Pages (considerando suas políticas). O formulário, se necessário, usa serviço de envio ou pequena função. **Por quê?** Simplicidade, baixa operação e arquivos globais.

2. Blog editorial ou documentação técnica. Hugo, Astro (SSG) ou MkDocs → build → estático/CDN. Previews via Git facilitam revisão. **Escala:** centenas de artigos ainda podem virar apenas arquivos.

3. Loja com WordPress/WooCommerce. Hospedagem WordPress/PHP bem dimensionada ou VPS/PaaS compatível + banco + backups + cache. **Por quê?** O ecossistema de extensões espera PHP e estado persistente. Pagamentos e segurança tornam os backups e atualizações essenciais.

4. Dashboard de controle de um bot. Frontend estático → API em VPS, container ou PaaS → banco. Processos contínuos do bot podem continuar na VPS. **Por quê?** Webhook, token e execução em segundo plano não pertencem ao navegador; não exigem que o frontend rode no mesmo host.

5. SaaS pequeno com usuários pagantes. Frontend na plataforma de escolha; backend persistente em PaaS; PostgreSQL gerenciado; object storage para anexos; monitoramento. **Por quê?** Evita operação desnecessária e mantém camadas claras. Reavalie funções para rotas de uso esporádico.

6. Página de campanha com picos de tráfego. Arquivos estáticos numa CDN; uma API leve separada se indispensável; proteção contra abuso. **Por quê?** Conteúdo cacheável suporta picos com pouca complexidade. A parte dinâmica costuma ser o gargalo.

7. API privada para automações. VPS com Nginx + backend; ou PaaS quando desejar terceirizar operação. Se os gatilhos forem raros e curtos, serverless pode ser excelente. **Por quê?** O padrão de uso decide mais do que a linguagem.

8. Plataforma global de produto com SSR, arquivos e filas. Frontend SSR em runtime compatível; CDN; banco gerenciado; object storage; filas/jobs. Deploy por Git e rollback. **Por quê?** A complexidade é justificada por usuários, equipe, SLA e necessidades mensuradas.

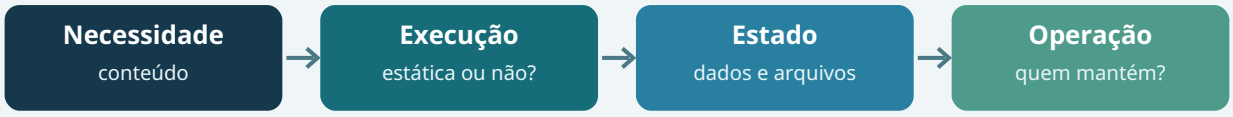
Como escolher entre duas opções boas

Pergunte: **qual delas a equipe consegue restaurar de madrugada?** Compatibilidade com framework, limites de execução, localização dos dados, observabilidade, lock-in, cobrança e segurança importam muito mais do que a frase “deploy em 30 segundos”.

IV · APLICANDO DE VERDADE

CAPÍTULO 16 / Seis caminhos e quando evitá-los

Matriz de decisão sem fanatismo



Decida primeiro o tipo de aplicação. A marca da plataforma vem depois.

Requisito dominante	Comece avaliando	Evite escolher automaticamente
HTML/CSS puro, docs, landing page	Static hosting / CDN	VPS complexa só por hábito
WordPress/PHP tradicional	Hospedagem PHP / VPS	CDN estática isolada
Backend sempre ativo / WebSocket	VPS, container, PaaS	Função com runtime incompatível
Pequenas rotas executadas raramente	Functions/serverless	Servidor 24 h sem necessidade
JS frontend + API própria	Frontend estático + API separada	Expor credenciais no frontend
Equipe grande, escala e compliance	Cloud gerenciada por componentes	Arquitetura distribuída sem métricas

Um algoritmo humano de decisão

Pergunta 1 — É só arquivo pronto? Sim → static hosting. Não → próxima.

Pergunta 2 — Precisa de PHP e CMS tradicional? Sim → hospedagem PHP/VPS adequada.

Pergunta 3 — Código executa o tempo todo, aceita conexões prolongadas ou precisa de SO específico? Sim → VPS, container ou PaaS permanente.

Pergunta 4 — Trabalho ocorre só por requisição ou evento, com limites conhecidos? Sim → avalie functions/serverless.

Pergunta 5 — Existem dados persistentes? Planeje banco/storage **separadamente** da camada executora.

Pergunta 6 — Quanto você quer operar? Muito → VPS; pouco → PaaS ou totalmente gerenciado. Ajuste pela dependência comercial, custos e requisitos.

Comparação pela responsabilidade

Modelo	Liberdade	Trabalho operacional	Escala típica
Compartilhada	Baixa/média	Baixo	Limitada ao plano
VPS	Alta	Alto	Manual/engenharia adicional
Site estático CDN	Alta para arquivos	Muito baixo	Alta para conteúdo cacheável
PaaS	Média/alta	Baixo/médio	Varia por serviço
Functions	Ajustada ao runtime	Baixo na infra	Automática dentro de cotas
Cloud modular	Muito alta	Médio/alto	Grande com boa arquitetura

Recomendação inicial: comece no modelo mais simples que cumpre requisitos *atuais*, e mantenha dados e deploys organizados para permitir uma futura migração.

V · OFICINAS PRÁTICAS

CAPÍTULO 17 / Do index.html ao servidor local, sem ferramenta mágica

Laboratório 1: um site estático no Linux Mint

Objetivo: ver a versão mínima da web em ação. Este laboratório precisa só de um terminal, navegador e editor de texto. Não cria serviço público.

1. Crie a pasta

```
mkdir -p ~/lab-hospedagem/site
cd ~/lab-hospedagem/site
```

Crie `index.html` com seu editor preferido:

```
<!doctype html>
<html lang="pt-BR">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Meu site 2026</title>
</head>
<body>
  <h1>Olá, web!</h1>
  <p>O princípio ainda é o mesmo.</p>
</body>
</html>
```

2. Rode um servidor de arquivos

```
python3 -m http.server 8000 --bind 127.0.0.1
```

Abra `http://127.0.0.1:8000/`. O Python está fazendo, em miniatura, o que um servidor HTTP faz: receber uma URL e devolver arquivos. O processo está preso ao loopback, então **não** virou hospedagem pública. Para interromper, use `Ctrl+C`.

3. Investigue com curl

```
curl -I http://127.0.0.1:8000/
curl http://127.0.0.1:8000/
```

Observe os headers e o conteúdo devolvido. Edite a página, recarregue e note que não existe build, cloud nem Git nesse caminho.

4. Experimento conceitual

Agora imagine copiar exatamente esses arquivos para um serviço estático. O que muda? A localização do servidor, DNS, HTTPS, disponibilidade e o processo de deploy. **O formato do arquivo servido não precisa mudar.**

Critério de sucesso: conseguir explicar o caminho `URL → servidor HTTP → arquivo → navegador` sem mencionar uma marca de hospedagem.

V · OFICINAS PRÁTICAS

CAPÍTULO 18 / Cloudflare Workers Static Assets e alternativas por Git

Laboratório 2: deploy estático moderno

Objetivo: publicar aquela mesma pasta `index.html` num serviço gerenciado. Aqui mostramos a rota atual recomendada pela documentação Cloudflare para novos projetos, além das rotas clássicas de Netlify/Pages.

Rota A: Workers Static Assets via CLI

Pré-requisitos: Node.js e npm compatíveis, conta Cloudflare com acesso à plataforma, conexão com a internet, arquivo `public/index.html` e Wrangler (a CLI oficial). O formato abaixo segue a configuração oficial de assets S24.

```
meu-site/
├── public/
│   └── index.html
└── wrangler.jsonc
```

Conteúdo de `wrangler.jsonc`:

```
{
  "name": "meu-site-estatico",
  "compatibility_date": "2026-10-16",
  "assets": { "directory": "./public" }
}
```

No diretório do projeto:

```
npm install --save-dev wrangler
npx wrangler login
npx wrangler dev
# Em outro terminal, experimente a URL local exibida.
npx wrangler deploy
```

A CLI informa o domínio de publicação, normalmente num subdomínio `workers.dev`, sujeito à configuração da conta. Você também pode associar domínio próprio. O tutorial oficial cobre o fluxo atualizado S23.

Rota B: Netlify com Git

1. Coloque `index.html` numa pasta/repositório Git.

2. No painel da Netlify, crie um projeto a partir do repositório.
3. Para HTML puro, não é obrigatório build; defina a pasta correta de publicação.
4. Faça `git push`, observe o deploy e visite a URL publicada.

A Netlify também documenta upload e deploy por CLI S07.

Rota C: Cloudflare Pages ou GitHub Pages

Cloudflare Pages segue operando com integração Git ou upload direto, embora Workers seja o caminho indicado para projetos **novos** S05, S02. GitHub Pages também funciona para site estático e documentação; não suporta PHP e não é serviço genérico para operar SaaS comercial S11, S12.

O que comparar nas três rotas

- Qual pasta foi publicada? Quem executou build, se houve?
- Como apareceu a URL pública e quem gerencia HTTPS?
- Como publicar uma segunda versão? Há prévia? É possível rollback?
- Há cota, licença ou proibição de uso comercial no plano escolhido?

Critério de sucesso: publicar a mesma página sem administrar uma máquina Linux pública. É a mesma web com um modelo operacional diferente.

V · OFICINAS PRÁTICAS

CAPÍTULO 19 / Da pasta local a um domínio real, controlado por você

Laboratório 3: seu próprio Nginx em uma VPS

Objetivo: recriar a hospedagem clássica usando uma VPS baseada em Ubuntu/Debian com Nginx. **Cuidado:** comandos administrativos são ilustrativos; revise políticas do seu provedor e não os execute em um host de produção com serviços existentes sem entender os impactos.

Pré-requisitos

VPS com acesso SSH autorizado, usuário com `sudo`, DNS do domínio sob seu controle, portas 80/443 liberadas no firewall e um backup antes de modificar configurações. Todos os `exemplo.com` abaixo são placeholders: substitua pelo seu domínio.

1. Instale o servidor

```
sudo apt update
sudo apt install nginx
sudo systemctl enable --now nginx
sudo systemctl status nginx
```

Visitar o IP em HTTP deve exibir uma resposta do Nginx, se firewall, nuvem e serviço estiverem corretos.

2. Crie a pasta que será servida

```
sudo mkdir -p /var/www/meusite
printf '<h1>Olá da VPS</h1>\n' | sudo tee /var/www/meusite/index.html
sudo chmod -R a+rX /var/www/meusite
```

3. Crie um virtual host

Crie `/etc/nginx/sites-available/meusite`:

```
server {
    listen 80;
    server_name exemplo.com www.exemplo.com;
    root /var/www/meusite;
    index index.html;
    location / { try_files $uri $uri/ =404; }
}
```

Ative e teste:

```
sudo ln -s /etc/nginx/sites-available/meusite \
  /etc/nginx/sites-enabled/meusite
sudo nginx -t
sudo systemctl reload nginx
```

Se há um virtual host default interferindo, revise a configuração e a prioridade. Não o remova às cegas em servidor compartilhado.

4. DNS e HTTPS

Crie registros DNS A/AAAA apropriados para apontar o domínio à VPS. Depois que o DNS propagar, configure TLS com Certbot ou outro mecanismo confiável. **Não copie comandos de instalação de certificados sem confirmar a versão do sistema**; use as instruções oficiais para Nginx S19.

5. Onde entra uma aplicação real?

Para Node/Python/Go, faça seu processo escutar apenas na rede interna quando possível, use `systemd` / container para gerenciá-lo e direcione Nginx via `proxy_pass` após ajustar headers e timeouts necessários. Para PHP, configure a integração com PHP-FPM de modo apropriado; **Nginx não interpreta PHP por conta própria**.

Critério de sucesso: enxergar a cadeia `DNS → IP → Nginx → pasta` e identificar quais responsabilidades agora são suas.

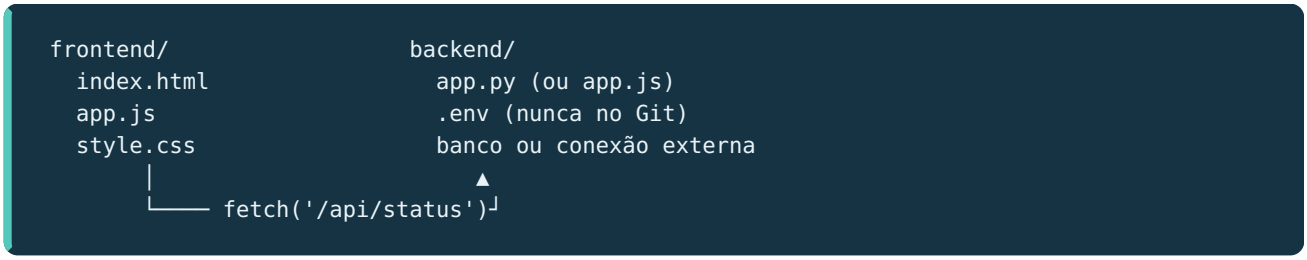
V · OFICINAS PRÁTICAS

CAPÍTULO 20 / Uma arquitetura que cresce sem virar Frankenstein

Laboratório 4: frontend + API, sem vazar segredos

Objetivo: entender a separação entre interface pública e serviço privado, sem depender de framework específico. Os trechos abaixo são esquemáticos e **não** constituem uma API pronta para expor à internet.

Estrutura lógica



Na prática, `fetch('/api/status')` funciona se o frontend e o backend estiverem sob a mesma origem com proxy configurado. Se estiverem em domínios diferentes, use URL explícita e controle CORS apropriadamente.

O que deve ficar no frontend

Formulários, layout, validações para UX, exibição de resultados e estado de interface. **Nunca** inclua tokens privados, senha do banco ou chave de API administrativa no bundle JavaScript. Arquivos publicados em CDN são públicos, mesmo que seus nomes estejam embaralhados.

O que deve ficar no backend

Autenticação e autorização, validações de segurança, rate limiting, acesso ao banco, uso de secrets, integração com serviços externos e processamento confiável de requisições.

Um request completo

1. O navegador carrega HTML/JS de uma CDN.
2. JavaScript chama `GET /api/status` no backend.
3. O backend autentica quando necessário e consulta a fonte de dados.
4. Retorna JSON sem segredos.
5. A interface exibe o resultado.

Onde cada parte pode viver?

Componente	Alternativa simples	Alternativa gerenciada
Frontend	Nginx na VPS	Workers Static Assets / Netlify
API	Serviço na VPS	PaaS / function compatível
Banco	PostgreSQL na VPS, com backup	Banco gerenciado
Arquivos de usuário	Diretório durável com backups	Object storage

Critério de sucesso: conseguir apontar, numa folha de papel, onde roda cada trecho de código e onde ficam dados e segredos. Esse exercício evita muitos problemas reais antes de escrever uma linha de framework.

VI · REFERÊNCIA DE MESA

CAPÍTULO 21 / Um mapa de troubleshooting sem adivinhação

Quando algo quebra: diagnóstico por sintomas

Sintoma	Causas frequentes	Primeira verificação
Domínio não abre	DNS, firewall, serviço offline	`dig`, `curl -I`, painel do provedor
HTTP funciona, HTTPS não	Certificado, porta 443, proxy	Validade TLS e vhost correto
Página aparece, JS/CSS falha	Paths errados, base URL, cache	Network tab do navegador
SPA dá 404 no refresh	Falta de fallback	Configuração de rotas estáticas
`502 Bad Gateway`	Backend inativo ou proxy errado	Logs Nginx + serviço de app
`500`	Exceção, env vars, permissão	Logs da aplicação
Deploy passou, página antiga	Cache, branch ou pasta publicada	Versão do deploy e cache CDN
Banco perde dados após deploy	Storage efêmero, volume incorreto	Persistência e backups
API falha só no browser	CORS, cookies, mixed content	Console e headers HTTP
Conta inesperadamente alta	Egress, invocações, DB, logs	Métricas e alertas de cobrança

Ferramentas úteis no Linux

```
curl -I https://exemplo.com
dig exemplo.com
sudo nginx -t
sudo journalctl -u nginx --since '1 hour ago'
docker compose ps
docker compose logs --tail=100 web
```

Os comandos específicos dependem do sistema e dos serviços. A ideia é inspecionar **uma camada por vez**: DNS → rede → TLS → proxy → runtime → dados → frontend.

O procedimento de emergência

1. Identifique a versão que falhou e o momento da mudança.

2. Veja status, logs e gráficos antes de alterar cinco coisas juntas.
3. Se necessário, execute rollback seguro para versão conhecida.
4. Confirme integridade e disponibilidade dos dados.
5. Corrija a causa e documente para a próxima ocorrência.

> **Dica de leitura:** `502` não significa necessariamente “Nginx quebrado”; muitas vezes ele está funcionando perfeitamente e informando que seu upstream não responde.

VI · REFERÊNCIA DE MESA

CAPÍTULO 22 / Trinta definições que você pode consultar sem sofrer

Glossário de sobrevivência web

API: interface para programas trocarem dados e operações. **Artefato:** saída pronta de uma build. **Backend:** código de servidor com regras, autenticação e dados. **Bucket:** recipiente lógico de objetos em storage.

Build: transformação do fonte em versão executável ou publicável. **Cache:** cópia para acelerar acessos. **CDN:** rede de distribuição e cache de conteúdo. **CI/CD:** automações de integração, testes e publicação.

Client-side: processamento realizado no navegador. **Container:** processo isolado com dependências empacotadas. **CORS:** mecanismo do navegador para controlar leitura entre origens. **CSR:** interface renderizada no cliente.

Deploy: colocar nova versão em produção. **DNS:** sistema que traduz nomes em informações de roteamento. **Edge:** infraestrutura distribuída perto da borda da rede. **Egress:** dados transferidos para fora de serviço/região conforme política de cobrança.

FTP/SFTP: transferência de arquivos; SFTP usa SSH. **Function:** unidade de código executada sob eventos/requisições numa plataforma. **Git:** sistema de versionamento. **HTTP:** protocolo de requisições e respostas.

HTTPS/TLS: HTTP protegido por criptografia e autenticação do servidor. **Object storage:** armazenamento de arquivos como objetos. **Origem:** serviço-fonte por trás de cache/CDN/proxy. **PaaS:** plataforma que executa aplicações gerenciando parte da infraestrutura.

Proxy reverso: ponto de entrada que encaminha a serviços internos. **Runtime:** ambiente de execução. **Serverless:** modelo que abstrai gestão/provisionamento da infraestrutura executora. **SPA:** app cliente que gerencia navegação na mesma página-base.

SSG: geração de HTML durante build. **SSR:** geração de HTML no servidor em resposta a requisições. **Stateful:** depende de dados/estado persistente; **stateless:** execução tratada como substituível entre chamadas. **VPS:** máquina virtual sob administração do cliente.

A tradução mais útil

> **“O projeto usa Next + Vercel + edge functions + Postgres + object storage.”**

Traduza para: “a interface usa um framework; o provedor publica/roda o app; parte do código executa em funções distribuídas; o banco guarda dados estruturados; os uploads ficam num serviço de arquivos”. Acabou o encanto do marketing. Agora são peças de engenharia.

VI · REFERÊNCIA DE MESA

CAPÍTULO 23 / Da teoria para deploys realmente seus

Roteiro de estudo de 30 dias

Não é necessário estudar tudo de uma vez. Uma sequência curta, prática e progressiva costuma dar resultados melhores do que seguir um roadmap de centenas de blocos.

Semana 1 — Fundamentos que nunca envelhecem

- Releia os capítulos 1–3. Monte um site HTML/CSS pequeno.
- Use `python3 -m http.server`, `curl` e DevTools/Network.
- Experimente rotas, código `404`, headers e cache do navegador.
- Estude DNS, A/AAAA, domínio e HTTPS em conceitos.

Entrega: site local e explicação desenhada do fluxo HTTP.

Semana 2 — O estilo de hospedagem dos anos 2020

- Versione site no Git com commits claros.
- Publique num serviço estático/Workers com deploy por CLI ou Git.
- Introduza build local com um gerador estático opcional.
- Crie uma branch e compare preview/deploy/rollback.

Entrega: site público com HTTPS e atualização reproduzível.

Semana 3 — Código no servidor e dados

- Monte uma API `GET /status` em sua linguagem favorita.
- Hospede-a localmente, depois em VPS ou PaaS.
- Faça frontend estático consumir a API.
- Introduza banco local ou persistência simples e um backup.

Entrega: projeto full-stack mínimo com segredo protegido.

Semana 4 — Operação responsável

- Instale um reverse proxy ou entenda seu equivalente gerenciado.
- Crie logs, health checks e monitoramento básico.
- Faça a primeira recuperação de backup em ambiente de teste.
- Compare custos e responsabilidade de VPS, PaaS e functions.

Entrega: README operacional com comandos de deploy, rollback e restauração.

VI · REFERÊNCIA DE MESA

CAPÍTULO 24 / A web moderna em dez princípios

Resumo de uma página: cole ao lado do monitor

1. **O navegador pede; alguma infraestrutura responde.** O princípio de 1999 ainda vale.
2. **Estático é arquivo pronto.** Ele pode ter sido escrito à mão ou gerado em build.
3. **Framework ≠ hospedagem.** React/Vue/Next.js respondem a perguntas diferentes de Nginx/Vercel.
4. **Git não serve páginas.** Ele costuma disparar publicação.
5. **CDN acelera distribuição.** Não transforma automaticamente conteúdo privado em seguro.
6. **Backend guarda a lógica sensível.** Secrets nunca devem entrar no frontend publicado.
7. **Estado persistente exige projeto próprio.** Não confie em disco temporário de container/função.
8. **Serverless ainda usa servidores.** Você terceiriza provisionamento e vive sob limites próprios.
9. **VPS dá poder e responsabilidade.** Backup, atualização, firewall e TLS ficam no seu colo.
10. **Escolha pelo requisito.** Arquitetura boa é a menor que resolve o problema com segurança.

A resposta direta à pergunta original

Sim, você ainda pode subir `index.html` e `.php` numa pasta. Pode usar Apache, Nginx, WordPress e SFTP como antes. O estilo moderno acrescentou pipelines de build, deploy por Git, redes de cache, bancos externos e runtimes gerenciados para melhorar distribuição e operação de aplicações maiores.

Se precisar começar agora: **site simples → estático/CDN; WordPress → PHP gerenciado; API/bot → VPS ou PaaS; função curta esporádica → serverless; caso grande → compor serviços depois de medir.**

> **O truque não é aprender 100 nomes. É saber, para cada nome, se ele serve um arquivo, executa código, guarda dados ou facilita o deploy.**

FONTES E LEITURAS COMPLEMENTARES

Biblioteca de referência

Links oficiais selecionados e verificados para a edição de outubro de 2026. Marcadores [S01] etc. ao longo do texto apontam para estas referências. Estas páginas podem evoluir depois da publicação.

S01 **MDN — O que é um servidor web?**
https://developer.mozilla.org/pt-BR/docs/Learn_web_development/Howto/Web_mechanics/What_is_a_web_server

S02 **Cloudflare — Pages, visão geral e recomendação para novos projetos**
https://developers.cloudflare.com/pages/

S03 **Cloudflare — Workers Static Assets**
https://developers.cloudflare.com/workers/static-assets/

S04 **Cloudflare — Boas práticas para Workers (outubro de 2026)**
https://developers.cloudflare.com/workers/best-practices/workers-best-practices/

S05 **Cloudflare — Pages: integração com Git**
https://developers.cloudflare.com/pages/configuration/git-integration/

S06 **Cloudflare — Pages: HTML estático**
https://developers.cloudflare.com/pages/framework-guides/deploy-anything/

S07 **Netlify — Criar deployments**
https://docs.netlify.com/deploy/create-deploys/

S08 **Vercel — Limites**
https://vercel.com/docs/limits

S09 **Vercel — Planos**
https://vercel.com/docs/plans

S10 **Next.js — Opções de deploy**
https://nextjs.org/docs/app/getting-started/deploying

S11 **GitHub — Começando com GitHub Pages**
https://docs.github.com/en/pages/getting-started-with-github-pages

S12 **GitHub — Limites e usos de GitHub Pages**
https://docs.github.com/pt/pages/getting-started-with-github-pages/github-pages-limits

S13 **Docker — O que é um container?**
https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-a-container/

S14 **Docker — Por que usar Compose?**
https://docs.docker.com/compose/intro/features-uses/

S15 **AWS — Hospedagem estática com Amazon S3**
https://docs.aws.amazon.com/pt_br/AmazonS3/latest/userguide/WebsiteHosting.html

S16 **AWS — Controle de acesso do S3 e CloudFront**
https://docs.aws.amazon.com/pt_br/AmazonS3/latest/userguide/access-management.html

S17 **AWS — O que é Lambda?**
https://docs.aws.amazon.com/lambda/latest/dg/welcome.html

S18 **AWS — O que é App Runner?**
https://docs.aws.amazon.com/apprunner/latest/dg/what-is-apprunner.html

S19 **Certbot — Instruções para Nginx**
https://certbot.eff.org/instructions?os=ubuntufocal&ws=nginx

S20 **Cloudflare — Limites de Workers**
https://developers.cloudflare.com/workers/platform/limits/

S21 **Netlify — Primeiros passos com Functions**
https://docs.netlify.com/build/functions/get-started/

S22 **Vercel — Política de uso comercial do Hobby**
https://vercel.com/docs/limits/fair-use-guidelines

S23 **Cloudflare — Workers Static Assets: tutorial**
https://developers.cloudflare.com/workers/static-assets/get-started/

S24 **Cloudflare — Workers: configuração de assets**
https://developers.cloudflare.com/workers/static-assets/binding/

S25 **AWS — S3 website endpoints e HTTPS**
https://docs.aws.amazon.com/AmazonS3/latest/userguide/WebsiteEndpoints.html

S26 **Netlify — Como publicar um site (agosto de 2026)**
https://www.netlify.com/knowledge-base/how-to-deploy-a-site-to-netlify/

S27 **AWS — Lambda Web Functions (outubro de 2026)**
https://docs.aws.amazon.com/lambda/latest/lambda-web/Welcome.html

Para aprender com material guiado

MDN Web Docs: base conceitual, sem depender de fornecedor. Mecânica da web.

Roadmap.sh: use as trilhas de Frontend, Backend e DevOps como mapa, não como checklist obrigatório. roadmap.sh.

Fireship: vídeos curtos para construir vocabulário; complemente sempre com documentação oficial. Canal Fireship.

Cloudflare Cache: bom aprofundamento em CDN, cache e diferença entre origin e edge. Getting Started with Cache.

Este guia é educativo e independente. Nomes de plataformas são exemplos de categorias, não patrocínios ou garantias de adequação. Antes de publicar sistemas reais, valide requisitos de segurança, contratos, localização de dados, licenças e cobrança do provedor escolhido.